Modelagem e Simulação

Resumo para estudo — Unidade 1 · Aulas 01, 02 e 03 · Prof. Luiz Batista Cardoso · Universidade Franciscana (UFN)

Conteúdo

1. Avaliação da disciplina
2. O que é modelagem e o processo de modelagem (7 etapas)
3. Sistemas discretos × contínuos
4. Tipos de modelagem: determinística, estocástica e baseada em agentes
5. Aplicações e ferramentas
6. Exemplo-guia: fila de carros no semáforo
7. Verificação × Validação
8. Técnicas de verificação e de validação
9. Quando o modelo falha: parâmetros, estrutura e extrapolação
10. Exemplos resolvidos: controle de estoque e circuito RC
11. Exercícios resolvidos (Aula 02/03)
12. Trabalho avaliativo — estrutura e critérios
13. Revisão rápida (flashcards) e referências

1. Avaliação da disciplina

- Exercícios e participação em aula — peso **2,0**
- Trabalho em aula com consulta — peso **3,0**
- Produto de Aprendizagem (avaliação) — peso **5,0**

$$NF = (N1 + N2 + N3) / 3$$

2. O que é modelagem

Modelar é construir uma representação simplificada de um sistema real (equações, regras, código) para estudá-lo, prever seu comportamento e testar cenários sem precisar mexer no sistema real. **Simular** é executar esse modelo ao longo do tempo (ou de outra grandeza) para observar os resultados. Exemplos de uso: lançamento de foguetes (SpaceX), aviação, análise de dados e projeto de produtos 3D.

Processo de modelagem — 7 etapas

#	Etapa	O que se faz
1	**Definição do problema**	Identificar o que queremos estudar e quais perguntas responder.
2	**Coleta e análise de dados**	Reunir informações sobre o sistema e identificar as variáveis importantes.
3	**Formulação matemática**	Traduzir o problema em equações e estabelecer simplificações (modelo conceitual).
4	**Implementação computacional**	Converter o modelo matemático em código executável (modelo computacional).
5	**Validação e calibração**	Comparar resultados simulados com dados reais e ajustar parâmetros.
6	**Análise de resultados**	Interpretar resultados, gerar visualizações e extrair insights.
7	**Refinamento e otimização**	Melhorar o modelo com base nos resultados e explorar cenários.

> **Pergunta-chave para escolher o tipo de modelo:** como o sistema que desejamos representar se comporta ao longo do tempo, do espaço ou de qualquer outra grandeza relevante? Ele muda **em saltos** (discreto) ou de forma **suave** (contínuo)?

3. Sistemas discretos × contínuos

Variáveis discretas

Assumem apenas valores contáveis (geralmente inteiros).

```
int numero_questoes = 12;
```

Nº de pessoas numa sala: 0, 1, 2, 3... nunca 1,5.

Variáveis contínuas

Assumem qualquer valor real dentro de um intervalo.

```
double media_notas = 6.0;
```

Temperatura, sinais/ondas, energia elétrica.

Sistemas discretos

- Valores específicos (inteiros)
- Mudanças em instantes pontuais (degraus)
- Descritos por **equações de diferenças**
- Tempo em intervalos discretos (n = 0, 1, 2...)

$$x[n+1] = f(\, x[n] \,)$$

Exemplos computacionais: redes de computadores (pacotes), sistemas de filas (atendimento), jogos digitais (estados), sinais digitais (DSP).

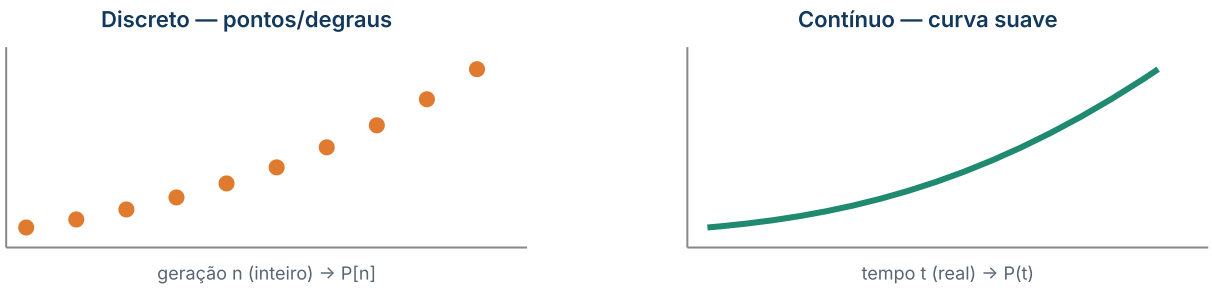
Sistemas contínuos

- Variáveis mudam de forma suave, sem saltos
- Qualquer valor dentro de um intervalo
- Descritos por **equações diferenciais**
- Tempo como variável contínua (t)

$$dy/dt = f(\, t, y \,)$$

Exemplos computacionais: dinâmica de fluidos (CFD), circuitos analógicos, controle industrial, fenômenos físicos (calor, ondas, luz).

Equação diferencial: relaciona uma função (posição, temperatura, velocidade...) com suas derivadas, isto é, com sua *taxa de variação* em relação ao tempo ou a outra variável. Descreve como uma quantidade muda continuamente.



Quadro-resumo

Aspecto	Modelo contínuo	Modelo discreto
Variável de estado	Números reais	Inteiros / estados finitos
Evolução	Equações diferenciais	Equações de diferenças / regras / eventos
Mudança de estado	Suave e gradual	Abrupta, em saltos
Simulação	Requer discretização numérica (passo Δt)	Direta, natural ao computador
Exemplos	Física, população, difusão	Filas, autômatos, saldo bancário

Modelos vistos em aula (com estrutura de classe)

Modelo	Tipo	Equação (forma usual)	Classe (atributos / métodos)
Crescimento populacional discreto (Malthus)	Discreto	`P[n+1] = P[n] + r·P[n]`	`ModeloMalthusDiscreto: populacao:int, taxa_crescimento:float, n:int · proximo_estado(), avancar()`
Crescimento populacional (Malthus)	Contínuo	`dP/dt = r·P → P(t)=P₀·e^(rt)`	`ModeloMalthus: populacao, taxa_crescimento, tempo (float) · derivada(), avancar(delta_t)`
Juros compostos (capital mês a mês)	Discreto	`C[n+1] = C[n]·(1 + i)`	`ModeloJurosCompostos: capital:float, taxa_juros:float, n:int · proximo_estado(), avancar()`
Resfriamento de um corpo (Lei de Newton)	Contínuo	`dT/dt = −k·(T − T_amb)`	`ModeloResfriamento: temperatura, temp_ambiente, k, tempo · derivada(), avancar(delta_t)`

> **Padrão para reconhecer no código:** modelos discretos têm `n:int` e `proximo_estado()`; modelos contínuos têm `tempo:float`, `derivada()` e `avancar(delta_t)` — o computador aproxima a equação diferencial em passos pequenos Δt (ex.: método de Euler: y(t+Δt) ≈ y(t) + Δt·f(t,y)).

Atenção: os juros compostos usam capital em reais (float), mas o modelo é **discreto** porque o *tempo* avança em saltos (mês a mês). O que define o tipo é como o estado evolui.

4. Tipos de modelagem

Determinística

Resultado único e previsível para um conjunto de entradas.

- Mesmas entradas → sempre a mesma saída
- Não há incerteza

Ex.: trajetória de um projétil.

Estocástica

Incorpora elementos aleatórios, produzindo diferentes resultados possíveis.

- Futuro previsto só com certa probabilidade
- Envolve variáveis aleatórias

Ex.: previsão meteorológica; Monte Carlo.

Baseada em agentes

Simula comportamentos individuais que geram padrões coletivos.

- **Autonomia:** cada agente age com regras próprias
- **Interação:** o sistema depende das trocas entre agentes e ambiente

Ex.: comportamento de multidões.

Na classificação de um modelo, responda a três eixos independentes: **discreto × contínuo**; **determinístico × estocástico**; **baseado em equações × baseado em agentes**.

5. Aplicações e ferramentas

Área	Aplicações
Indústria	Otimização de linhas de produção, fluxos industriais, análise de confiabilidade, design de produtos virtuais
Medicina	Propagação de epidemias, descoberta de medicamentos, fluxo sanguíneo, dinâmica molecular
Jogos	Física realista, simulação de multidões, efeitos visuais, inteligência artificial
Finanças	Risco financeiro, simulação Monte Carlo (valores aleatórios), análise de mercados, otimização de portfólios

Ferramenta	Uso
SimPy (Python)	Simulação de eventos discretos com processos e `env.timeout()` (ex.: carro alterna estacionar/dirigir).
AnyLogic	Plataforma visual multimétodo: tráfego, logística, portos, estações, armazéns.
VPython	Simulação física e visualização 3D (partículas, molas).
SUMO	Simulation of Urban Mobility — tráfego urbano e redes viárias.
PyMOL / Schrödinger	Visualização e modelagem molecular (proteínas, DNA, fármacos).

```
import simpy
def carro(env):
    while True:
        print(f"Começou a estacionar em {env.now}")
        yield env.timeout(5)   # duração estacionado
        print(f"Começou a dirigir em {env.now}")
        yield env.timeout(2)   # duração da viagem
```

6. Exemplo-guia: fila de carros no semáforo

Exemplo usado nas três aulas (código em *github.com/luizbcardoso/modelagem-simulacao*). Veículos se acumulam esperando o sinal verde.

$$\text{Fila}(t+1) = \text{Fila}(t) + \text{Chegadas} - \text{Saídas}$$

- Sinal **VERMELHO**: Saídas = 0
- Sinal **VERDE**: Saídas = mínimo(Fila, capacidade de passagem)
- Classe `ModeloFilaTransito`: `fila, chegadas, saida, n : int` · `proximo_estado()`, `avancar()`
- Gráfico em degraus: a fila sobe no vermelho, desce no verde → modelo **discreto e determinístico**.

```
# Lógica de saída de carros
if sinal == "VERMELHO":
    saida = 0
else:
    saida = saida_por_segundo
# Equação de estado: Fila nova = Fila antiga + Chegadas - Saídas
fila = fila + (chegadas_por_segundo - saida)
# Regra: fila não fica negativa
if fila < 0:
    fila = 0
```

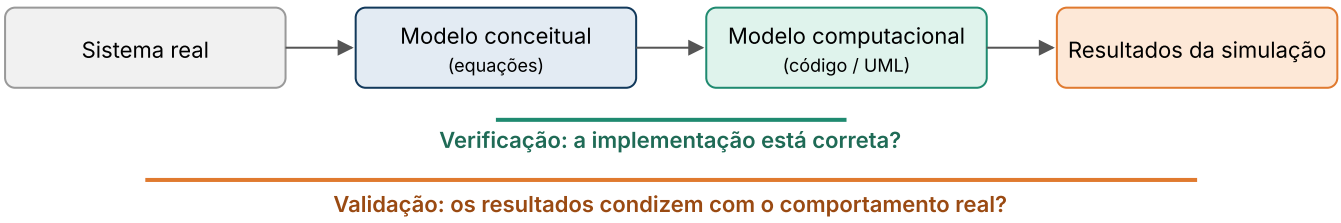
A versão **contínua** do exemplo modela a **aceleração** dos carros ao abrir o sinal (velocidade varia suavemente, integrada a cada passo Δt = 0,5 s).

7. Verificação × Validação

Pergunta central da Aula 02: **como sabemos se um modelo está correto?** São duas perguntas diferentes:

VERIFICAÇÃO — "o modelo foi construído corretamente?" (código)
Confirmar que o **modelo computacional foi implementado corretamente**, isto é, que o código faz exatamente o que a equação do **modelo conceitual** determina.

VALIDAÇÃO — "construí o modelo correto?" (realidade)
Confirmar que o **modelo conceitual representa adequadamente o sistema real** que se pretende estudar.



Um modelo pode estar **perfeitamente verificado e ainda assim ser inválido**. No exemplo da fila, o código segue fielmente *Fila(t+1) = Fila(t) + Chegadas − Saídas* → **verificado** ✓. Mas ele representa uma rua real (ex.: R. Silva Jardim)? Os valores, o número de faixas, os tipos de veículo e a capacidade correspondem à realidade? → **validado?** Só dados reais respondem.

8. Técnicas

Técnicas de verificação testam o código

Técnica	Como funciona
Comparação com solução analítica	Quando a equação diferencial ou de diferença tem solução fechada, comparar a saída da simulação com essa solução exata. É a forma mais direta. A diferença deve se limitar à precisão do método numérico.
Testes de casos-limite	Testar situações simples e previsíveis em que a saída é totalmente conhecida. Ex.: se a taxa de chegada for zero, a fila deve permanecer exatamente constante.
Correspondência de lógica e equação	Garantir que o código respeita as regras do modelo e impede valores fisicamente impossíveis (fila/estoque/população negativos, capacitância ≤ 0).

Técnicas de validação comparam com a realidade

Validar é, em geral, **mais difícil** que verificar: exige dados do mundo real e "adequadamente" depende do **propósito** do modelo.

Técnica	Como funciona
Comparação com dados históricos/empíricos	Confrontar a saída com medições reais (fluxo médio de carros em um horário, histórico de vendas).
Validação de face	Mostrar o comportamento do modelo a especialistas no domínio (engenheiro de tráfego, gestor de estoque) e perguntar se é plausível.
Análise de sensibilidade	Variar sistematicamente os parâmetros e ver se a resposta muda de forma plausível e proporcional. Um modelo populacional que "explode" com pequena variação de r indica problema estrutural.
Validação preditiva (aprendizado de máquina)	Calibrar parâmetros com parte dos dados históricos e testar a capacidade de prever a outra parte (não usada na calibração) — como treino/teste.

9. Quando um modelo falha: ajuste e iteração

É comum — e esperado — que o modelo não passe de primeira. Investigar três causas:

Causa	Pergunta	Sintoma	Correção
1. Parâmetros	Os parâmetros foram bem calibrados?	A estrutura da equação está certa, mas os valores numéricos não refletem o fenômeno real.	Calibrar a partir de dados observados, **sem alterar a estrutura**.
2. Estrutura	A estrutura do modelo está completa?	O modelo ignora um fator relevante (ex.: epidemia sem imunidade adquirida).	Revisar o modelo conceitual: novas variáveis de estado ou termos.
3. Extrapolação	Estamos dentro do limite para esta simulação?	Modelo validado para uma faixa de condições sendo usado fora dela.	Reconhecer os limites de validade e evitar extrapolações não testadas.

Aplicado à fila de carros:

- **Parâmetros:** 3 carros entrando e 2 saindo por segundo refletem um semáforo real? Um erro de medição levaria a uma representação errada.
- **Estrutura:** há via dupla? motos? só um tipo de veículo? Falta algo no modelo?
- **Limites:** sem capacidade definida, rodar 3 horas gera uma fila que a rua não comporta — a simulação "não tem limites".

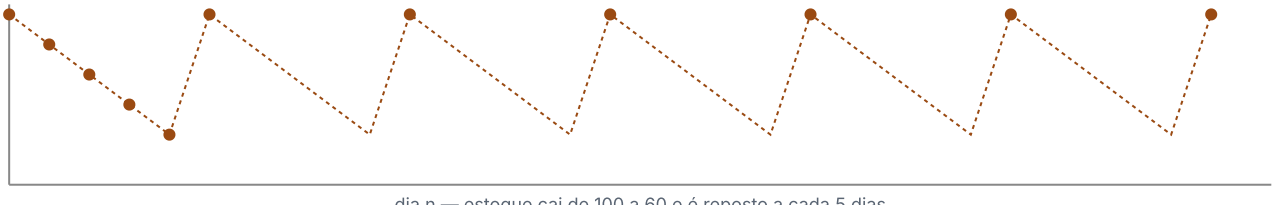
V&V é um **ciclo iterativo**, não uma etapa única: sempre que o modelo é ajustado, deve ser **verificado e validado novamente**. Esse refinamento contínuo é marca de um trabalho de modelagem maduro.

10. Exemplos resolvidos (Aula 03)

A) Controle de estoque — discreto, determinístico, baseado em equação

O nível de estoque S diminui a cada dia n com a demanda e é reposto periodicamente → padrão em **"dente de serra"**. O estado só faz sentido como inteiro medido no fim de cada dia.

$$S[n] = S[n-1] - \text{demanda_diária} + R(n)$$



dia n — estoque cai de 100 a 60 e é reposto a cada 5 dias

Variável	Tipo
Estoque S[n]	Discreto
Demanda diária	Discreto
Intervalo de reposição (n)	Discreto

Verificação

- **Casos-limite:** demanda nula e sem reposição → estoque estritamente constante.
- **Solução analítica:** com consumo e intervalos constantes, calcular a curva teórica e comparar ponto a ponto.
- **Lógica e equação:** subtrair vendas e somar reposição só nos dias certos, com inteiros, sem saldo negativo.

Validação

- **Dados empíricos:** confrontar com registros reais de inventário e vendas de um armazém em um mês.
- **Face:** especialista em logística confirma taxa de consumo e picos de reposição.
- **Sensibilidade:** simular oscilações de demanda ou atrasos e ver se prevê falta/excesso de forma coerente.
- **Preditiva:** calibrar com período anterior e prever o seguinte.

B) Circuito RC — contínuo, determinístico, baseado em equação

Descreve como a tensão/carga em um capacitor evolui ao ser carregado por uma fonte através de um resistor. Variável de estado: carga (ou tensão); parâmetros: R, C e tensão da fonte. Base de

temporizadores e filtros.

$$Q(t) = Q_{max} \cdot (1 - e^{-t/RC}) \quad \text{com} \quad Q_{max} = C \cdot V_{fonte}$$

Q(t): carga em coulombs · e ≈ 2,718 · t em segundos · R em ohms · C em farads. O produto **RC** é a constante de tempo: após 1·RC atinge ~63%; após ~5·RC, praticamente 100% (curva tende assintoticamente à tensão da fonte).

Verificação

- **Casos-limite:** para t → ∞, a corrente no capacitor deve estabilizar em zero.
- **Solução analítica:** comparar a integração numérica com a solução fechada; diferença limitada à precisão do método.
- **Lógica e equação:** impedir C negativo ou nulo.

Validação

- **Dados empíricos:** medir carga de um capacitor físico com osciloscópio e sobrepor à curva simulada.
- **Face:** engenheiro eletricista avalia curvas transitórias (ex.: cerâmico × supercapacitor).
- **Sensibilidade:** variar ESR e temperatura; ripple e dissipação devem mudar de forma lógica e proporcional.
- **Preditiva:** calibrar ESR/ESL em baixa frequência e prever impedância em alta frequência, comparando com medição não usada na calibração.

11. Exercícios resolvidos

1) Fila contínua (aceleração): uma técnica de verificação e uma de validação.

Verificação **Caso-limite:** com aceleração e frenagem = 0 m/s², a velocidade inicial deve permanecer exatamente constante durante toda a simulação.

Validação **Dados históricos:** coletar telemetria de veículos reais partindo do zero em um semáforo e comparar a curva de velocidade simulada com a medida.

2) Cenário de falha para parâmetro, estrutura e extrapolação.

- **Parâmetro:** aceleração calibrada em 30 m/s² (irreal para um carro). O código calcula certo, mas o valor não reflete o sistema → compromete a **validação**.
- **Estrutura:** modelo omite arrasto aerodinâmico e perda de tração/torque em marchas altas (aceleração estática) → compromete a **validação** (fatores reais ignorados no modelo conceitual).
- **Extrapolação:** sem velocidade máxima, ou com sinal verde por horas, o carro chegaria a velocidades absurdas → compromete a **validação** (fora dos limites de projeto).

3) E se Δt for reduzido de 0,5 s para 0,01 s?

Não causa falha. Pelo contrário, **aumenta a precisão** da integração numérica: passos menores reduzem o erro de discretização das equações diferenciais. A entrada/saída de carros não muda conceitualmente — apenas é calculada com mais resolução (o custo é mais processamento).

4) Verificado mas não validado × validado mas não verificado (exemplo: vendas de uma padaria).

- **Verificado, não validado:** *Vendas = Habitantes × Taxa* implementada perfeitamente, mas ignora chuva, horários de pico e obras na rua → previsões que não condizem com a realidade.
- **Validado, não verificado:** um bug subtrai impostos indevidamente e outro soma um “bônus fantasma”; os erros se anulam e a curva casa com os dados reais. A saída atende às expectativas, mas o código não corresponde ao modelo matemático.
- **Pode persistir sem ser percebido?** Sim — enquanto os resultados baterem com os dados, ninguém desconfia. O problema aparece quando as condições mudam (ex.: mudança na alíquota) e os erros

deixam de se compensar. Por isso a verificação deve ser feita mesmo quando a validação “passa”.

12. Trabalho: Modelagem, Verificação e Validação

- Escolher um **sistema simples** de uma empresa ou da Computação e construir uma pequena simulação computacional.
- Arquivo **PDF**, individual ou em dupla, entregue no **Minha UFN**.
- Sistema real **próprio e inédito** — **não** usar exemplos de aula (fila de carros, estoque, RC, Malthus, juros, resfriamento).
- Prazo informado: 12 de outubro, 23h59.

Seção do PDF	Conteúdo exigido
1. Descrição e contextualização (2,0)	Explicar o fenômeno real. Tabela de variáveis: **nome**, **tipo** (discreto/contínuo), **papel** (variável de estado, parâmetro, entrada ou saída).
2. Formulação e classificação (2,0)	Equação ou regra de transição (equação de diferença p/ discreto; diferencial p/ contínuo). Classificação justificada: discreto × contínuo; determinístico × estocástico; agentes × equações.
3. Plano de verificação (2,0)	Pelo menos **UM** teste: casos-limite, correspondência de lógica ou solução analítica.
4. Plano de validação (2,0)	Pelo menos **DUAS** estratégias: dados empíricos/históricos, validação de face, análise de sensibilidade.

Critérios de correção (10,0): definição do problema e coleta de dados (2,0) · classificação do sistema (2,0) · lógica computacional e regras de transição (2,0) · plano de V&V (2,0) · **análise de falhas potenciais** (2,0 — parâmetro, estrutura, extrapolação). Relacionado ao Produto de Aprendizagem I.

13. Revisão rápida

Pergunta	Resposta
Discreto ou contínuo — como decidir?	Pelo modo como o estado evolui: em saltos/instantes contáveis (discreto) ou de forma suave (contínuo).
Equação típica de cada um?	Discreto: x[n+1] = f(x[n]) (diferenças). Contínuo: dy/dt = f(t,y) (diferencial).
Por que o contínuo precisa de Δt?	O computador é discreto; a equação diferencial é aproximada em passos. Δt menor → mais precisão, mais custo.
Determinístico × estocástico?	Mesma entrada → mesma saída × incorpora aleatoriedade (resultado probabilístico).
Características de agentes?	Autonomia (regras próprias) e interação (com outros agentes e ambiente).
Verificação compara o quê com o quê?	Modelo computacional (código) × modelo conceitual (equações).
Validação compara o quê com o quê?	Modelo/resultados × sistema real.
Qual é mais difícil e por quê?	Validação: exige dados reais e depende do propósito do modelo.
3 técnicas de verificação	Solução analítica, casos-limite, correspondência de lógica/equação.
4 técnicas de validação	Dados históricos, validação de face, análise de sensibilidade, validação preditiva.
3 causas de falha	Parâmetros mal calibrados, estrutura incompleta, extrapolação fora dos limites.
Parâmetro errado: corrige mudando a estrutura?	Não — recalibra com dados observados, mantendo a estrutura.
V&V é feita uma vez?	Não — é um ciclo iterativo: a cada ajuste, verificar e validar de novo.

Referências

PIDD, Michael. *Modelagem empresarial: ferramentas para tomada de decisão*. Porto Alegre: Bookman, 1998.
PRADO, Darci Santos do. *Teoria das filas e da simulação*. Belo Horizonte: INDG, 1999.
CAMPOS, Vicente Falconi. *Usando o Arena em simulação*, 2000.
Material complementar: Profa. Dra. Laura Vendrame; Prof. Dr. André Flores dos Santos; Prof. Dr. Mirkos Ortiz Martins.